“TRAVELMATE AI: SMART TRAVEL COMPANION AND GUIDANCE PLATFORM”

A PROJECT REPORT

Submitted in partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

IN

ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

Submitted by

Gangavaram Joshika

24STUCHH010445

Under the Supervision of

Dr V V SATYANARAYANA MURTHY B

As part of the Internship Program - I

Department of Artificial Intelligence And Machine Learning

ICFAI Foundation for Higher Education, Hyderabad, India

June 2026

Certificate of Approval

This is to certify that the project report titled "TRAVELMATE AI: SMART TRAVEL COMPANION AND GUIDANCE PLATFORM" is a bonafide record of work carried out by Gangavaram Joshika (Roll No: 24STUCHH010445) under my supervision and guidance. The work described in this report is submitted in partial fulfillment of the requirements for the award of the Internship-1 for the degree of Bachelor of Technology in Artificial Intelligence and Machine Learning from ICFAI Foundation for Higher Education, Hyderabad, and has not been submitted elsewhere for any other degree or diploma.

_____

Internal Supervisor

Dr V V SATYANARAYANA MURTHY B
Department of AI & ML

_____

Head of Department

Faculty of Science & Technology
ICFAI Foundation for Higher Education

Acknowledgement

I express my deepest gratitude to my esteemed project supervisor, Dr V V SATYANARAYANA MURTHY B, Department of Artificial Intelligence and Machine Learning, for his invaluable guidance, continuous encouragement, and constructive feedback throughout the course of this project. His expertise and insightful suggestions have played a critical role in shaping the architecture and implementation of this platform.

I would also like to thank the Head of the Department and the administration of IcfaiTech, Faculty of Science & Technology, ICFAI Foundation for Higher Education, Hyderabad, for providing the necessary academic resources, infrastructure, and an encouraging environment that enabled the successful execution of this internship project.

Finally, I extend my heartfelt appreciation to my family and peers for their constant support, patience, and motivation, which helped me stay focused and overcome technical challenges during the development of TravelMate AI.

Gangavaram Joshika
Roll No: 24STUCHH010445

Abstract

Modern tourism relies heavily on digital information, yet travelers frequently face the challenge of fragmented data across multiple websites when planning a trip. Information regarding regulatory checklists, visa requirements, local traffic and criminal laws, cultural etiquette, local cuisines, and accommodation options must be compiled manually. Furthermore, budget-conscious travelers require customized, structured itineraries, while international tourists benefit from context-aware, real-time, multilingual guidance.

To address these challenges, this project presents TravelMate AI, a comprehensive, Streamlit-based web application designed to serve as an intelligent, all-in-one travel companion. The platform integrates a local SQLite database pre-seeded with regulatory, geographical, and cultural profiles for multiple destinations. The frontend is built using Streamlit, styled with custom CSS injections and Outfit typography. Key modules include a Home Dashboard with global search, a Country Guide displaying pre-departure checklists and cultural dos/don'ts, a City Explorer showcasing rated attractions, cuisines, and hotel tiers, and a Smart Travel Planner that dynamically generates day-by-day itineraries (1-7 days) paired with interactive Plotly budget charts.

A core innovation of the platform is the context-aware AI Travel Assistant, a conversational chatbot. Using the Google Agent Development Kit (google-adk) and the gemini-2.5-flash model, the chatbot autonomously executes SQL database tool queries. When API access is unavailable, the system gracefully degrades to an offline, rule-based keyword matching parser that utilizes active UI session contexts (selected country/city) to resolve ambiguous queries. Security is maintained through JWT-based session tokens and SHA-256 password hashing. The codebase enforces strict quality standards via automated Pytest suites and pre-commit hooks covering formatting, security (Bandit), and static type checks (Mypy). TravelMate AI represents a scalable, local-first solution that enhances traveler safety, cultural compliance, and planning efficiency.

Table of Contents

1. Introduction

1.1 Project Overview

In an increasingly globalized world, tourism has evolved from a luxury into a highly accessible activity. However, planning international or domestic travel remains a complex and fragmented process. Travelers must research visa requirements, local emergency services, transit systems, accommodation options, and dining details. Beyond logistics, a significant challenge is understanding local laws and cultural etiquette, which vary drastically across countries. For instance, regulations regarding jaywalking, littering, or public consumption of certain items carry heavy fines in places like Singapore, while cultural norms regarding tipping or dress codes dictate social interactions in Japan.

TravelMate AI is a comprehensive, local-first web application built on the Streamlit framework, designed to serve as an intelligent, all-in-one travel companion. By combining structured relational data with context-aware artificial intelligence, TravelMate AI consolidates visa guidelines, regulatory warnings, cultural etiquette, local transit guides, and hotel listings into a single responsive interface. The platform also features a dynamic, budget-oriented travel planner and a multi-lingual AI chatbot that acts as an autonomous database agent.

1.2 Problem Statement

Traditional travel planning is highly fragmented, requiring tourists to visit multiple independent web portals to compile an itinerary, verify visa rules, list local laws, and locate emergency numbers. This manual compilation is time-consuming and prone to errors. Existing travel platforms (such as TripAdvisor and Google Travel) excel at business recommendations but fail to provide centralized regulatory, legal, and safety checklists. Additionally, static itinerary planning does not adapt dynamically to custom durations or budget tiers, and most platforms lack context-aware, real-time conversational assistants that operate locally. There is a clear need for a unified, secure, and multi-lingual system that provides structured destination profiles alongside intelligent, context-sensitive chat assistance.

1.3 Objectives

The primary objectives of the TravelMate AI project are:

- **1. Centralize Travel Information:** To design and implement a relational SQLite database storing regulatory, visa, safety, and cultural guidelines alongside city-specific attractions, hotels, dining, and transit options.
- **2. Provide Dynamic Itinerary Planning:** To develop a customizable planning module that generates day-by-day timelines for 1-7 days, paired with interactive Plotly budget visualizations.
- **3. Implement Context-Aware Conversational AI:** To integrate an autonomous database agent using the Google Agent Development Kit (google-adk) and Gemini models, with a robust rule-based keyword fallback that utilizes UI session states.
- **4. Support Multi-lingual Accessibility:** To build internationalization (i18n) support, offering complete UI and database translation in English, Hindi, and Telugu.

- **5. Secure User Data:** To establish JWT-based session management and SHA-256 password hashing for user registration, profile editing, and travel history persistence.
- **6. Enforce Software Quality:** To implement a comprehensive BDD-style test suite using Pytest and integrate pre-commit hooks for code style, static analysis, and security scanning.

2. Literature Survey & Gap Analysis

2.1 Existing Travel Guidance Systems

Digital travel assistance has seen significant growth, with platforms categorized into commercial review portals, governmental advisory sites, and automated itinerary generators. Commercial platforms like TripAdvisor and Yelp rely on user-generated content, offering excellent restaurant and hotel reviews. However, they lack structured legal and cultural guidance. Government portals (such as the US Department of State Travel Advisories) provide excellent safety and visa updates but are text-heavy, lack local city guides, and offer no planning utilities. Automated planning apps (like Sygic Travel) offer day-by-day mapping but require paid subscriptions, lack local regulatory context, and do not integrate conversational AI agents.

2.2 Comparison of Travel Platforms

The following table compares the features of TravelMate AI with major existing digital travel platforms:

Feature / Metric	TripAdvisor	Google Travel	Govt. Advisories	TravelMate AI (Proposed)
Centralized Visa & Regulatory Info	No (Fragmented)	No	Yes (Text-only)	Yes (Structured Table)
Local Law & Etiquette Alerts	No	No	Yes (General)	Yes (Specific / Detailed)
Dynamic Budget Visualizer	No	No (Flights/Hotels only)	No	Yes (Interactive Plotly Chart)
Context-Aware AI Chatbot	No	Yes (Gemini - General)	No	Yes (Autonomous DB Agent & Fallback)

2.3 Research Gap

A critical analysis of existing systems reveals a major research gap: the lack of integration between commercial tourism listings and regulatory/cultural safety guidelines. Tourists often travel without knowing local restrictions or cultural norms, leading to legal penalties or social discomfort. Furthermore, standard itinerary planners do not provide instant, context-sensitive chat support linked to the user's active search context. TravelMate AI bridges this gap by combining regulatory country profiles, localized city explorer tables, dynamic budgeting, and an autonomous AI chatbot that inherits the active UI selection as its conversational context.

3. System Design and Architecture

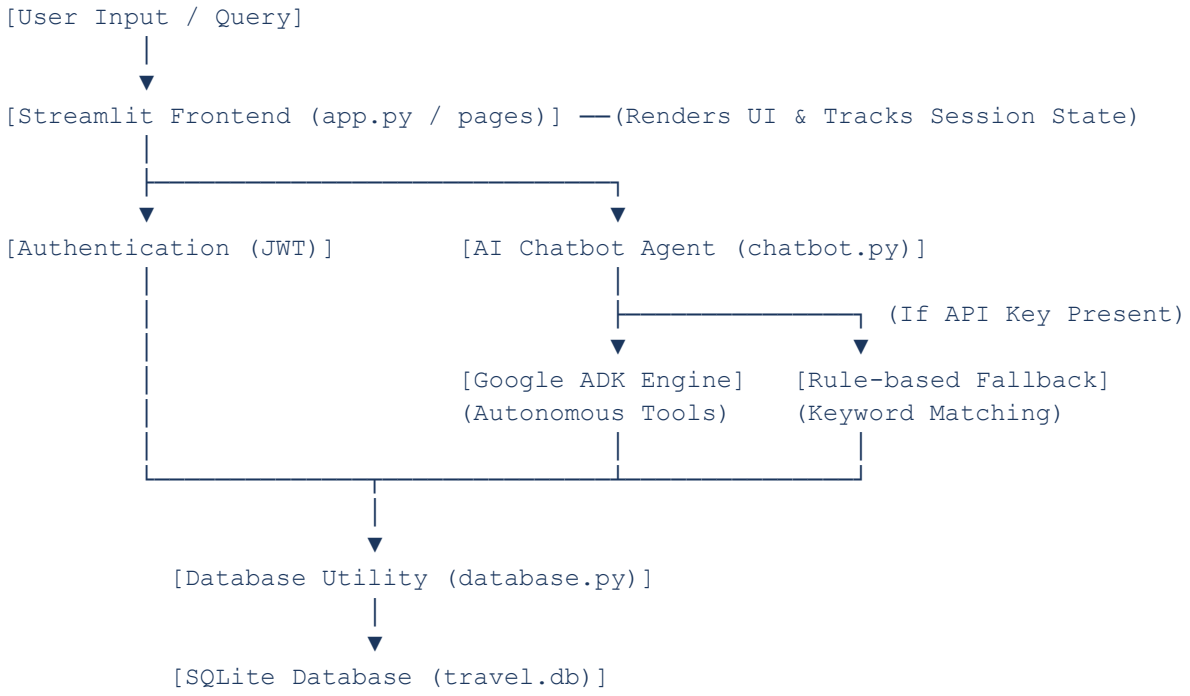
3.1 Architectural Overview

TravelMate AI implements a decoupled three-layer architecture to ensure modularity, scalability, and ease of maintenance. The system separates user interaction, business/agent logic, and data storage into distinct layers:

- **1. Presentation Layer (Frontend):** Built using Streamlit. It consists of multiple view files (home, country_info, city_info, planner, chatbot, profile, history, saved_trips, auth) that render dynamic UI components, charts, and input controls. Custom CSS is injected to enforce the 'Outfit' typography and a modern dark/light card-based aesthetic.
- **2. Application / Logic Layer (Backend):** Handles the core processing, including JWT-based session management, password cryptography (SHA-256), multi-lingual translation dictionary lookups, and the AI chatbot orchestration. The chatbot module integrates the Google Agent Development Kit (google-adk) to coordinate autonomous tool calls or execute the rule-based keyword matching algorithm.
- **3. Data Access Layer (Database):** Consists of an embedded SQLite database (travel.db) managed via Python's sqlite3 library. This layer executes connection pooling, parameterized SQL queries, and transactional updates for user profiles, search histories, and saved itineraries.

3.2 System Workflow Flowchart

The flowchart below describes the step-by-step data and logic flow when a user interacts with the platform:



3.3 Advantages of the Proposed System

- **- Consolidated Information:** Combines logistical, regulatory, and cultural data in a single dashboard.
- **- Dynamic Planning & Visualization:** Generates detailed, day-by-day schedules with interactive budget breakdowns.
- **- Context-Aware Conversational Interface:** The AI assistant automatically inherits the active UI search context, minimizing user input.
- **- Multi-lingual Localization:** Complete localization in English, Hindi, and Telugu expands accessibility.
- **- Local-First & Highly Secure:** An embedded SQLite database requires zero configuration, while JWTs and password hashing ensure user data privacy.

4. Database Design

The relational database schema is implemented in SQLite (travel.db). It is designed to maintain strict data integrity and support fast query execution. The database consists of six tables: countries, cities, users, travel_history, saved_trips, and weather_history. The schema details for each table are described below:

4.1 Table: countries

Column Name	Data Type	Description
id	INTEGER PRIMARY KEY	Unique country identifier, auto-incremented
country_name	TEXT UNIQUE	Name of the country (e.g., India, Japan, Singapore)
capital	TEXT	Capital city of the country
currency	TEXT	Local currency symbol and code (e.g., INR, JPY, SGD)
language	TEXT	Primary official language(s) spoken
timezone	TEXT	Time zone offset (e.g., GMT+9, GMT+8)
emergency_number	TEXT	Emergency contact numbers (police, medical, fire)
visa_info	TEXT	Visa requirements and pre-departure checklists
rules	TEXT	Crucial local laws and regulatory warnings
etiquette	TEXT	Cultural dos and don'ts for travelers
safety_tips	TEXT	General safety recommendations and crime warnings

4.2 Table: cities

Column Name	Data Type	Description
id	INTEGER PRIMARY KEY	Unique city identifier, auto-incremented
country_id	INTEGER	Foreign Key referencing countries(id) with Cascade Delete

Column Name	Data Type	Description
city_name	TEXT	Name of the city (e.g., Tokyo, Osaka, Hyderabad)
description	TEXT	Brief geographical and historical overview
transport_info	TEXT	Local transit details (buses, trains, subways)
food_info	TEXT (JSON)	JSON array of local delicacies and descriptions
tourist_places	TEXT (JSON)	JSON array of attractions, ratings, and best visit times
hotel_info	TEXT (JSON)	JSON object of hotels categorized by price tier
shopping_areas	TEXT	Popular local markets, shopping streets, and malls
airport_details	TEXT	Local airport names and transfer guidelines
safety_recommendations	TEXT	Neighborhood-specific safety advisories

4.3 Table: users

Column Name	Data Type	Description
id	INTEGER PRIMARY KEY	Unique user identifier, auto-incremented
full_name	TEXT	Full name of the registered user
email	TEXT UNIQUE	Unique email address used for login
phone	TEXT	Contact phone number
country	TEXT	User's country of residence
city	TEXT	User's city of residence
profile_pic	TEXT	Base64 encoded string of profile picture or image URL
password_hash	TEXT	SHA-256 hashed password string
preferences	TEXT (JSON)	JSON array of user travel preferences (e.g., vegetarian, luxury)

4.4 Table: travel_history

Column Name	Data Type	Description
id	INTEGER PRIMARY KEY	Unique history record identifier
user_id	INTEGER	Foreign Key referencing users(id)
activity_type	TEXT	Type of activity ('search', 'itinerary', 'chat')
query	TEXT	User input query or destination searched
details	TEXT (JSON)	JSON string containing metadata or generated itinerary details
is_favorite	INTEGER	Binary flag (0 or 1) indicating if the item is bookmarked

4.5 Table: saved_trips

Column Name	Data Type	Description
id	INTEGER PRIMARY KEY	Unique saved trip identifier
user_id	INTEGER	Foreign Key referencing users(id)
trip_type	TEXT	Type of saved item ('itinerary', 'destination', 'hotel')
name	TEXT	Name of the saved trip or destination
collection_name	TEXT	Name of the folder/collection (defaults to 'My Saved Trips')
details	TEXT	JSON string containing complete details of the saved item
travel_date	TEXT	Planned travel date in YYYY-MM-DD format

4.6 Table: weather_history

Column Name	Data Type	Description
id	INTEGER PRIMARY KEY	Unique weather record identifier
city_id	INTEGER	Foreign Key referencing cities(id)
month_num	INTEGER	Month number (1 = January, 12 = December)

Column Name	Data Type	Description
month_name	TEXT	Full name of the month (e.g., January)
avg_temp	REAL	Average monthly temperature in degrees Celsius
rainfall	REAL	Average monthly rainfall in millimeters
description	TEXT	General weather description (e.g., Hot & Humid)

5. Modules Description

5.1 Home Dashboard & Search

The Home Dashboard serves as the primary landing page. It features a modern hero header and showcases featured destination cards (such as Hyderabad, Singapore, and Tokyo). A global search bar allows users to type queries like 'shrine', 'biryani', or country/city names. The search engine queries multiple columns across both countries and cities tables, returning matching destinations with direct links to their profiles. A sidebar dropdown allows users to select an active country and city, which sets the global context across the application.

5.2 Country Information Guide

The Country Guide provides regulatory, legal, and cultural information. It displays a profile summary including the capital, currency, languages, timezone, and local emergency contact numbers. A key feature is the Pre-departure Checklist, which lists visa requirements and essential travel items. The module also contains a 'Rules & Etiquette' section detailing critical local laws (e.g., bans on chewing gum in Singapore or traffic regulations in Japan) and cultural dos and don'ts to ensure respectful travel.

5.3 City Explorer

The City Explorer offers localized tourist recommendations. It is divided into four sections: (1) Sightseeing & Attractions: displays popular spots, their star ratings, and the best time of day to visit; (2) Stays & Accommodations: lists hotels categorized into Budget, Mid-range, and Luxury tiers with estimated prices; (3) Cuisines & Dining: showcases traditional vegetarian and non-vegetarian dishes; (4) Transit Guidelines: provides local transportation details and airport transfer options.

5.4 Smart Travel Itinerary Planner

The Itinerary Planner allows users to customize their travel schedules. Users select a trip duration (1-7 days) and a budget level (Economy, Mid-range, Luxury). The system dynamically generates a day-by-day timeline featuring morning, lunch, afternoon, and evening activities. Simultaneously, it renders an interactive Plotly Pie Chart representing the cost allocation across accommodation, dining, transit, and shopping, accompanied by an itemized budget table. Registered users can save these itineraries to their profile.

5.5 AI Travel Assistant Chatbot

The AI Chatbot provides real-time conversational assistance. It operates in two modes: (1) Agent Kit Mode: if a Gemini API key is configured, it utilizes the Google Agent Development Kit to run an autonomous agent that calls database query tools; (2) Offline Mode: if the API key is absent, it falls back to a local rule-based keyword matching parser. The chatbot is context-aware: if a user asks a general question like 'where should I stay?', the bot automatically retrieves the active country or city selected in the sidebar session state.

5.6 User Authentication & Profile

The Authentication module manages user accounts. Users can register, log in, and secure their sessions using a 'Remember Me' token. The system generates a JWT-like token using HMAC-SHA256 and hashes passwords locally using SHA-256 with a static salt. Once logged in, users can edit their profile details, view dashboard analytics (total trips planned, countries explored, and AI usage stats), view their search history, and access saved itineraries.

6. Implementation Details

6.1 Key Code Structure

The codebase is structured to enforce separation of concerns. The entry point is app.py, which configures the multi-page router. View components are stored in the pages/ directory, database operations are defined in utils/database.py, cryptographic functions are in utils/auth_utils.py, and global CSS injections are managed by utils/styles.py. Internationalization (i18n) is handled by utils/i18n.py, which stores translation mappings for English, Hindi, and Telugu UI elements.

6.2 Main Entry and Navigation

The navigation and routing are implemented using Streamlit's st.Page and st.navigation APIs. The sidebar dynamically updates the available pages based on the user's login status. If a user is logged in, their profile, travel history, and saved trips are added under an 'Account' group, and their name, email, and profile picture are displayed in a styled header card. If the user is a guest, an 'Auth' page is displayed instead.

6.3 JWT Authentication and Session Management

Password security is implemented using SHA-256 hashing with a static salt to prevent rainbow table attacks. Session persistence is achieved by generating a signed JWT-like token containing the user's ID, name, email, and an expiration timestamp. The token is signed using HMAC-SHA256 with a secret key. When the application starts, it checks the session state for a 'remember_me_token'. If present and valid, it decodes the token and automatically logs the user in, restoring their session without requiring re-authentication.

7. Algorithms & AI Integration

7.1 Autonomous AI Agent (Google ADK)

The AI Travel Assistant integrates the Google Agent Development Kit (google-adk) and the gemini-2.5-flash model. The agent is defined with a natural language instruction set and is equipped with three python functions registered as tools: get_country_info, get_city_info, and search_destinations. When the user submits a chat query, the Agent Runner analyzes the request and autonomously decides which tool to execute, parses the tool's output, and formats a coherent markdown response.

```
travel_agent = Agent(
    name="travel_mate_agent",
    model="gemini-2.5-flash",
    instruction="You are TravelMate AI. Use the database tools provided
(get_country_info, get_city_info, search_destinations) to answer user queries. If the
query does not mention a location, refer to the Active Context in the system
message.",
    tools=[get_country_info, get_city_info, search_destinations]
)
```

7.2 Rule-Based Context-Aware Fallback

To ensure the application remains functional offline or when an API key is not configured, a local rule-based fallback algorithm is implemented in pages/chatbot.py. The algorithm operates as follows:

- **1. Tokenization & Normalization:** The user's query is normalized (converted to English using i18n helpers) and tokenized into lowercase words.
- **2. Entity Recognition:** The tokens are matched against country and city names in the database. If a match is found, that entity is set as the target. If no match is found, the system retrieves the active country/city from the UI session state (context fallback).
- **3. Topic Detection:** The query is checked for specific keyword sets. For example, keywords like 'food', 'cuisine', or 'biryani' trigger the 'Food' topic; 'rules', 'law', or 'chewing gum' trigger the 'Rules' topic; 'hotel' or 'stay' trigger the 'Accommodations' topic.
- **4. Database Querying & Formatting:** Based on the detected topic and target entity, the system executes the corresponding SQL query, parses the JSON payload (if applicable, such as food_info or tourist_places), and formats a structured response in the user's selected language.

8. Application Interface Screenshots

This section contains screenshots of the working TravelMate AI application, demonstrating its user interface, multi-page layout, and features.

Figure 1: TravelMate AI Application Banner and Branding

Figure 1 shows the branding banner for TravelMate AI. The design features a modern, clean aesthetic with the slogan 'Smart Travel Companion'. It is used at the top of the Home page and on the cover page to establish a premium visual identity.

Figure 2: City Explorer - Hyderabad Page

Figure 2 illustrates the City Explorer page when 'Hyderabad' is selected. The interface displays detailed local profiles including sightseeing spots (such as Charminar and Golconda Fort) with star ratings and recommended times to visit. It also lists local cuisines (e.g., Hyderabadi Biryani, Double ka Meetha) and categorizes accommodations into budget, mid-range, and luxury tiers.

Figure 3: City Explorer - Singapore Page

Figure 3 shows the City Explorer page for 'Singapore'. This view highlights local transit guidelines, airport details for Changi Airport, and city safety recommendations. It provides international travelers with essential information on navigating the city-state's public transport systems.

Figure 4: City Explorer - Tokyo Page

Figure 4 showcases the City Explorer page for 'Tokyo'. It details traditional and modern attractions (such as Senso-ji Temple and Shibuya Crossing), local dining options, and popular shopping areas (like Ginza and Akihabara). The layout uses structured cards and custom CSS styling.

9. Testing & Quality Assurance

9.1 BDD-style Pytest Suite

The project implements a BDD-style (Behavior Driven Development) unit test suite using Pytest. The tests are organized into modules verifying specific components of the system. Mocking is utilized for database connections and API calls to ensure independent, fast test execution. The table below lists the test suites and the behaviors they verify:

Test Suite File	Scope of Verification	Example Test Cases
test_database_spec.py	Database connection, initialization, and queries	Verifies database seeding, country/city lookups, and blank search query fallbacks.
test_auth_spec.py	User registration, login, and JWT cryptography	Verifies password hashing, JWT encoding/decoding, and token expiration validation.
test_chatbot_spec.py	Chatbot query parsing and context resolution	Verifies keyword matching, active context fallback, and response formatting.
test_history_spec.py	Activity logging and travel history management	Verifies history item logging, favorite toggling, and history clearing.
test_i18n_spec.py	UI translation and internationalization	Verifies language mapping, key translations, and multi-lingual UI rendering.
test_styles_spec.py	CSS injection and HTML rendering utilities	Verifies custom style injection and markup safety checks.
test_weather_spec.py	Weather history lookups and recommendations	Verifies monthly weather retrieval and packing recommendations.
test_agent_spec.py	AI Agent tool calling and Runner operations	Verifies ADK agent initialization, tool execution, and query coordination.
test_app_spec.py	Main router configuration and page loading	Verifies page routing, sidebar rendering, and session state initialization.

9.2 Quality Assurance and Pre-commit Hooks

To maintain high code quality, security, and formatting consistency, the project enforces strict pre-commit hooks and CI/CD pipeline checks. These include:

- **- Code Formatting:** Ruff-format is used to maintain a consistent code style across all python files.

- **- Static Analysis & Linting:** Ruff, Pylint, Flake8, and Mypy (type-checking) are executed to identify syntax issues, code smells, and type mismatches.
- **- Security Auditing:** Bandit scans the codebase for common security flaws (e.g., hardcoded secrets or unsafe imports), and pip-audit checks third-party packages for known vulnerabilities.
- **- Secret Scanning:** Gitleaks is integrated to prevent accidental commits of API keys, passwords, or tokens.
- **- CI/CD Pipeline:** A GitLab CI/CD pipeline (.gitlab-ci.yml) is configured to run all lints, security scans, and Pytest suites automatically on every commit.

10. Advantages, Limitations, & Future Scope

10.1 Advantages

- **- Unified Platform:** Consolidates visa checklists, local laws, cultural etiquette, transit guides, hotel listings, and dining options into a single portal.
- **- Dynamic Planning:** Generates customized itineraries with interactive cost breakdowns, eliminating static planning limitations.
- **- Context-Aware AI Chatbot:** Maintains active country/city selections as conversational context, reducing the need for repetitive user inputs.
- **- Multi-lingual UI:** Complete localization in English, Hindi, and Telugu ensures accessibility for a wider range of users.
- **- Secure & Local-First:** Uses an embedded SQLite database and local JWT session management, ensuring high performance and privacy.

10.2 Limitations

- **- Limited Destination Data:** The database is currently pre-seeded with only three countries (India, Japan, Singapore). Expanding the data requires manual seeding or external API integrations.
- **- API Dependency for Advanced AI:** The autonomous agent mode requires an active internet connection and a Google Gemini API key; otherwise, the system falls back to a simpler keyword-based matching engine.
- **- Informational Only:** The application provides hotel, dining, and transit recommendations but does not support direct bookings or live reservations.

10.3 Future Scope

- **- Geographical Expansion:** Integrating external travel APIs (e.g., TripAdvisor or Google Places API) to dynamically fetch information for any country or city worldwide.
- **- Live Booking Integration:** Partnering with flight and hotel booking APIs (e.g., Amadeus or Booking.com) to allow users to book flights, hotels, and activities directly from the app.
- **- Real-Time Weather Forecasting:** Replacing historical weather averages with live weather forecasting APIs (e.g., OpenWeatherMap) to provide real-time packing suggestions.
- **- Voice-Enabled AI Assistant:** Integrating speech-to-text and text-to-speech capabilities into the chatbot to assist travelers on the go.
- **- Cross-Platform Mobile Application:** Developing a mobile version of TravelMate AI using React Native or Flutter to provide offline access to saved itineraries and emergency guides.

11. Conclusion

TravelMate AI successfully addresses the challenges of fragmented information and lack of cultural awareness in travel planning. By consolidating visa requirements, local regulations, cultural etiquette, and city recommendations into a single multi-lingual platform, the application provides tourists with a reliable, structured, and easy-to-use companion. The integration of a dynamic itinerary planner with interactive budget charts allows for customized trip scheduling. Furthermore, the context-aware AI chatbot, powered by the Google Agent Development Kit and Gemini, provides intelligent conversational assistance, while the local rule-based fallback ensures uninterrupted service. Enforced by BDD-style testing and strict pre-commit quality standards, TravelMate AI is a secure, robust, and scalable solution that significantly enhances the travel experience, ensuring safety, cultural compliance, and planning efficiency.

References

[1] Streamlit Documentation, "Multi-page Apps and Navigation Configuration," [Online]. Available: https://docs.streamlit.io.

[2] SQLite Consortium, "SQLite Database Engine and SQL Syntax Reference," [Online]. Available: https://sqlite.org.

[3] Google AI, "Agent Development Kit (ADK) Developer Guide," [Online]. Available: https://github.com/google/adk.

[4] Plotly Technologies, "Plotly Open Source Graphing Library for Python," [Online]. Available: https://plotly.com/python.

[5] Pytest Developer Team, "Pytest: Helps you write better programs," [Online]. Available: https://docs.pytest.org.

[6] Internet Engineering Task Force (IETF), "RFC 7519: JSON Web Token (JWT) Specification," [Online]. Available: https://tools.ietf.org/html/rfc7519.

[7] Python Software Foundation, "The Python Standard Library - hashlib and hmac modules," [Online]. Available: https://docs.python.org/3/library.